Лабораторна робота № 5

Географічні дані OpenStreetMap: від тегів до маршруту

Інтелектуальний аналіз та візуалізація даних · 4 години · 4 бали

Постановка

Ви — аналітик у туристичному застосунку. Завдання: для заданого міста **відібрати десять найвартісніших історико-культурних об'єктів** і **спланувати піший маршрут**, який починається на залізничному вокзалі, обходить усі десять і повертається на вокзал.

Слово «найвартісніші» в даних не записане. Критерій ви побудуєте самі — і захистите його.

Потім те саме для **тридцяти** об'єктів. Результат цієї, більшої задачі стане базою порівняння в **лабораторній роботі № 8**, де ту саму задачу розв'язуватимуть генетичним та мурашиним алгоритмами.

Що здаєте

1. Цей ноутбук із виконаними комірками (усі виводи збережені).
2. Файл `lr5_результати.json` — результати для ЛР8.
3. Карту `lr5_маршрут.html`.
4. Звіт у кінці ноутбука: відповіді на сім питань.

Що оцінюється

критерій	0	2	3	4
Відбір об'єктів	теги взято навмання	фільтр працює	фільтр обґрунтовано	показано, що відкинуто і чому
Критерій рейтингу	немає	є формула	ваги пояснено	порівняно з альтернативним критерієм
Маршрут	не будується	будується	перевірено коректність	пояснено різницю методів числом
Карта	немає	точки на карті	маршрут по ребрах графа	видно різницю з прямими відрізками

критерій	0	2	3	4
Звіт	немає	формально	з числами	з висновком, якого не було в завданні

0. Ваш варіант

Впишіть прізвище та номер у списку групи. Місто підтягнеться само.

In [1]:
```
STUDENT_NAME = 'Михайло Б'     # напр. 'Коваленко Іван Петрович'
LIST_NUMBER  = 2      # номер у списку групи

CITIES = [
    'Львів', 'Київ', 'Харків', 'Одеса', 'Дніпро',
    'Чернівці', 'Івано-Франківськ', 'Тернопіль', 'Ужгород', 'Луцьк',
    'Рівне', 'Хмельницький', 'Вінниця', 'Житомир', 'Чернігів',
    'Черкаси', 'Полтава', 'Суми', 'Кропивницький', 'Запоріжжя',
    'Миколаїв', 'Кривий Ріг', 'Кам\u2019янець-Подільський', 'Мукачево',
    'Коломия', 'Дрогобич', 'Біла Церква', 'Ніжин', 'Острiг', 'Умань',
]

assert STUDENT_NAME and LIST_NUMBER > 0, 'впишіть прізвище та номер у списку
CITY = CITIES[(LIST_NUMBER - 1) % len(CITIES)] + ', Україна'
print('варіант %d: %s' % (LIST_NUMBER, CITY))
```

```
варіант 2: Київ, Україна
```

> Якщо викладач призначив вам інше місто — просто перепишіть `CITY` вручну і зазначте це у звіті.

In [2]:
```
import importlib.util, subprocess, sys
REQUIRED = ['osmnx', 'geopandas', 'shapely', 'pyproj', 'pyarrow',
            'folium', 'mapclassify', 'h3', 'networkx']
missing = [p for p in REQUIRED if importlib.util.find_spec(p) is None]
if missing:
    print('встановлюю:', ', '.join(missing), '...')
    subprocess.check_call([sys.executable, '-m', 'pip', 'install', '-q', *mi
print('усі пакети на місці')
```

```
усі пакети на місці
```

In [3]:
```
import warnings; warnings.filterwarnings('ignore')
import json, time, os, collections
import numpy as np
import pandas as pd
import geopandas as gpd
import networkx as nx
import matplotlib.pyplot as plt
import folium
from xyzservices import providers as xyz
```

```
BASEMAP = xyz.Esri.WorldGrayCanvas   # CARTO з 2026 р. вимагає ключ API
pd.set_option('display.width', 200)
print('geopandas', gpd.__version__)
```

```
geopandas 1.2.0
```

---

1. Два способи дістати дані

Робоча програма вимагає порівняти два способи доступу **за обсягом переданих даних і за часом**:

- **живий запит** — `osmnx` питає Overpass API просто зараз;
- **знімок** — ті самі дані, збережені у файли GeoParquet, читаються з диска.

Нижче — живий запит. Запустіть його **один раз**: він збереже теку зі знімком, і всі наступні запуски читатимуть з неї.

> Overpass — безкоштовний сервіс і обмежує частоту. Якщо запит не пройшов — зачекайте хвилину й повторіть. Якщо не проходить взагалі, візьміть резервний знімок у викладача і вкажіть це у звіті.

In [5]:
```
import osmnx as ox

OUT = 'osm_' + CITY.split(',')[0].strip().lower().replace(' ', '_')
os.makedirs(OUT, exist_ok=True)
ox.settings.use_cache = True
ox.settings.cache_folder = os.path.join(OUT, 'cache')
ox.settings.requests_timeout = 300
ox.settings.overpass_rate_limit = True
ox.settings.overpass_url = 'https://overpass-api.de/api'

TOURISM_OK = ['museum', 'attraction', 'gallery', 'artwork', 'viewpoint']

def cache_bytes():
    import pathlib
    p = pathlib.Path(ox.settings.cache_folder)
    return sum(f.stat().st_size for f in p.glob('*.json')) if p.exists() els

print('тека знімка:', OUT)
```

```
тека знімка: osm_київ
```

Дві налаштовані дрібниці, без яких запит не пройде

Це не формальності — кожна з них ламає збір по-своєму, і жодна не схожа на свою справжню причину.

Представитись. Overpass відповідає **HTTP 406** клієнтові, який прийшов із типовим заголовком `python-requests`. `osmnx` надсилає свій `User-Agent` сам, але якщо ви звертаєтесь до сервісу напряму — надсилайте теж. Рядок має бути **лише латиницею**: HTTP-заголовки передаються в latin-1, і кирилиця кладе запит помилкою `UnicodeEncodeError` ще до відправлення.

Обрати машину. За іменем `overpass-api.de` стоїть кілька серверів, і частина з них відмовляє у з'єднанні. `osmnx` закріплює одну адресу на всю сесію — навмисно, щоб не порушувати облік слотів. Якщо закріпиться та, що відмовляє, впаде весь збір. Тому адресу обираємо самі, вже перевірену.

In [6]:
```
import socket

USER_AGENT = ('LvivPolytechnic-SK26-coursework/1.0 '
              '(https://lpnu.ua; student lab assignment)')
assert USER_AGENT.isascii(), 'HTTP-заголовки — latin-1, кирилиця тут не пра
ox.settings.http_user_agent = USER_AGENT


def pin_working_ip(hostname, port=443, timeout=8):
    """Закріпити для цього імені адресу, яка справді відповідає."""
    try:
        infos = socket.getaddrinfo(hostname, port, socket.AF_INET, socket.SO
    except Exception as e:
        print('не вдалось резолвити %s: %s' % (hostname, type(e).__name__))
        return None
    ips = sorted({i[4][0] for i in infos})
    good = []
    for ip in ips:
        s = socket.socket(); s.settimeout(timeout)
        try:
            s.connect((ip, port)); good.append(ip)
        except Exception:
            pass
        finally:
            s.close()
    print('%s: машин %d, відповідають %d %s' % (hostname, len(ips), len(gooc
    if not good:
        return None
    original = socket.gethostbyname
    socket.gethostbyname = lambda h: good[0] if h == hostname else original(
    print('закріплено', good[0])
    return good[0]


pin_working_ip('overpass-api.de')
```

```
overpass-api.de: машин 2, відповідають 2 ['162.55.144.139', '65.109.112.52']
закріплено 162.55.144.139
```

Out[6]:
```
'162.55.144.139'
```

> Якщо жодна машина не відповіла — виконайте комірку ще раз за хвилину, або візьміть у викладача резервний знімок і зазначте це у звіті. Інші дзеркала перелічені на сторінці Overpass API; адресу можна підставити у `ox.settings.overpass_url`.

Завдання 1.1

Допишіть функцію `fetch`: вона має зберегти шар у `.parquet`, а якщо файл уже є — прочитати його, **не звертаючись до мережі**. Поверніть також, скільки МБ пішло мережею і скільки секунд це зайняло.

Підказка щодо `clean`: у OSM одна колонка може містити і число, і список (ребро, склеєне з кількох шляхів). Parquet вимагає один тип на колонку, тому неоднорідну колонку треба звести до тексту **цілком**.

In [7]:
```
def clean(gdf):
    g = gdf.reset_index()
    for c in g.columns:
        if c == 'geometry':
            continue
        boxed = g[c].map(lambda v: isinstance(v, (list, dict, set, tuple))).
        if boxed or g[c].dtype == object:
            g[c] = g[c].map(
                lambda v: None if v is None or (isinstance(v, float) and pd.
                else (json.dumps(v, ensure_ascii=False)
                      if isinstance(v, (list, dict, set, tuple)) else str(v)
            ).astype('string')
    return g


COST = {}

def fetch(name, getter):
    """Повертає шар. Якщо файл уже збережено — читає з диска."""
    path = os.path.join(OUT, name + '.parquet')
    if os.path.exists(path) and os.path.getsize(path) > 0:
        t0 = time.time()
        g = gpd.read_parquet(path)
        COST[name] = {'спосіб': 'знімок', 'записів': len(g),
                      'передано_МБ': 0.0,
                      'файл_МБ': round(os.path.getsize(path) / 1e6, 3),
                      'секунд': round(time.time() - t0, 3)}
        return g

    b0 = cache_bytes()
    t0 = time.time()
    g = clean(getter())
    g.to_parquet(path, index=False)
    COST[name] = {'спосіб': 'живий запит', 'записів': len(g),
                  'передано_МБ': round((cache_bytes() - b0) / 1e6, 3),
```

```
                    'файл_МБ': round(os.path.getsize(path) / 1e6, 3),
                    'секунд': round(time.time() - t0, 3)}
    return g
```

In [8]:
```
boundary = fetch('boundary', lambda: ox.geocode_to_gdf(CITY))
poly = boundary.geometry.iloc[0]
print('межа міста отримана, площа %.1f км²'
      % (boundary.to_crs(32635).area.iloc[0] / 1e6))
```

```
межа міста отримана, площа 826.8 км²
```

In [11]:
```
heritage = fetch('heritage', lambda: ox.features_from_polygon(
    poly, {'historic': True, 'tourism': TOURISM_OK}))
stations = fetch('stations', lambda: ox.features_from_polygon(
    poly, {'railway': ['station', 'halt'], 'building': ['train_station']}))
amenities = fetch('amenities', lambda: ox.features_from_polygon(
    poly, {'amenity': ['cafe', 'restaurant', 'drinking_water', 'toilets']}))

# Мережа приходить двома таблицями одразу, тому їй окрема гілка.
NODES_P = os.path.join(OUT, 'road_nodes.parquet')
EDGES_P = os.path.join(OUT, 'road_edges.parquet')

if os.path.exists(NODES_P) and os.path.exists(EDGES_P):
    t0 = time.time()
    nodes, edges = gpd.read_parquet(NODES_P), gpd.read_parquet(EDGES_P)
    COST['мережа walk'] = {'спосіб': 'знімок', 'записів': len(nodes) + len(e
                           'передано_МБ': 0.0,
                           'файл_МБ': round((os.path.getsize(NODES_P)
                                             + os.path.getsize(EDGES_P)) / 1
                           'секунд': round(time.time() - t0, 3)}
else:
    t0, b0 = time.time(), cache_bytes()
    G_raw = ox.graph_from_polygon(poly, network_type='walk')
    nodes, edges = ox.convert.graph_to_gdfs(G_raw)
    clean(nodes).to_parquet(NODES_P, index=False)
    clean(edges).to_parquet(EDGES_P, index=False)
    nodes, edges = gpd.read_parquet(NODES_P), gpd.read_parquet(EDGES_P)
    COST['мережа walk'] = {'спосіб': 'живий запит', 'записів': len(nodes) +
                           'передано_МБ': round((cache_bytes() - b0) / 1e6,
                           'файл_МБ': round((os.path.getsize(NODES_P)
                                             + os.path.getsize(EDGES_P)) / 1
                           'секунд': round(time.time() - t0, 1)}

print('вузлів %d | ребер %d' % (len(nodes), len(edges)))
```

```
вузлів 178332 | ребер 520960
```

Завдання 1.2

Зведіть `COST` у таблицю і дайте відповідь у звіті: **у скільки разів читання знімка швидше за живий запит** і **скільки мегабайтів ви не передали мережею** при повторному запуску.

Контрольна ознака правильності: при **другому** запуску ноутбука всі рядки мають мати спосіб «знімок», а `передано_МБ` — нуль.

In [12]:
```
display(pd.DataFrame(COST).T)
```

	спосіб	записів	передано_МБ	файл_МБ	секунд
boundary	живий запит	1	0.054	0.049	2.284
heritage	знімок	2749	0.0	0.559	0.15
stations	знімок	149	0.0	0.104	0.063
amenities	живий запит	3732	1.396	0.46	82.543
мережа walk	живий запит	699292	72.299	35.936	234.7

> **Чому пішохідна мережа, а не автомобільна.** У старому місті музеї стоять за сотню метрів один від одного, а автомобільних вузлів там одиниці. Кілька пам'яток сядуть на **один** вузол графа, і матриця віддалей скаже, що між ними нуль метрів. Задача розв'яжеться, відповідь буде неправильною, і повідомлення про помилку не з'явиться. У кроці 4 ви це перевірите явно.

---

2. Хто претендує на звання пам'ятки

У OpenStreetMap немає тега «визначна пам'ятка». Є два сімейства:

- `historic=*` — `castle`, `church`, `monument`, `memorial`, `ruins`, `tower`, `yes`…
- `tourism=*` — `museum`, `gallery`, `attraction`, `artwork`, `viewpoint`

Перетин неповний: музей у сучасній будівлі має лише `tourism`, руїни фортеці — лише `historic`. Тому беремо **об'єднання**.

📖 https://wiki.openstreetmap.org/wiki/Map_Features

Завдання 2.1

Відберіть кандидатів і **назвіть у звіті, скільки об'єктів ви відкинули й чому**.

In [53]:
```
hist  = heritage['historic'].notna() if 'historic' in heritage else pd.Serie
tour  = heritage['tourism'].isin(TOURISM_OK) if 'tourism' in heritage else p
named = heritage['name'].notna() if 'name' in heritage else pd.Series(False,

mask = hist | tour
cand = heritage[mask & named].reset_index(drop=True)
```

```
print('усього у шарі heritage :', len(heritage))
print('підходять за тегами    :', int(mask.sum()))
print('з них мають name       :', int((mask & named).sum()))
print('відкинуто без тегів    :', int((~mask).sum()))
print('відкинуто без name     :', int((mask & ~named).sum()))
print('разом відкинуто        :', len(cand) - len(heritage))
```

```
усього у шарі heritage : 2749
підходять за тегами    : 2749
з них мають name       : 2156
відкинуто без тегів    : 0
відкинуто без name     : 593
разом відкинуто        : 593
```

In [54]:
```
assert 'cand' in dir(), 'спершу виконайте завдання 2.1'
print('=== historic ===');  print(cand['historic'].value_counts().head(10).t
print()
print('=== tourism ===');   print(cand['tourism'].value_counts().head(10).to
print()
print('типи геометрії:', cand.geometry.geom_type.value_counts().to_dict())
```

```
=== historic ===
historic
memorial    969
aircraft    102
ruins        53
cannon       49
building     45
tank         44
vehicle      41
monument     35
yes          15
heritage     13

=== tourism ===
tourism
artwork        386
attraction     242
museum         164
viewpoint       29
gallery         22
information      2
no               2
memorial         1
yes              1

типи геометрії: {'Point': 1895, 'Polygon': 257, 'LineString': 3, 'MultiPolyg
on': 1}
```

> **Перевірте себе.** Серед типів геометрії мають бути і `Point`, і `Polygon`. Якщо самі лише полігони — ви відфільтрували зайве: пам'ятники, меморіали та оглядові точки в OSM є **вузлами**, у них немає будівлі.

Якщо кандидатів менше тридцяти

Невелике місто може дати 15–20 об'єктів. Тоді:

1. розширте перелік `tourism` (наприклад, додайте `theatre`, `place_of_worship`);
2. або зменште `N_LARGE` до того, що є;

і **обов'язково зазначте це у звіті** разом із числом.

---

3. Критерій «найкращі»

Ключовий крок роботи. Стовпця «цінність» у даних немає. Його треба сконструювати і захистити.

Сигнали, які є в OSM:

сигнал	що означає
`wikidata` / `wikipedia`	про об'єкт є стаття — хтось сторонній визнав його вартим опису
кількість колонок `name:*`	назву переклали N мовами — перекладають те, що шукають іноземці
`tourism=museum/gallery`	призначений для відвідування
`historic` заповнений	має історичну природу
площа полігона	розмір — **найслабший сигнал**: великим буває і склад

Завдання 3.1

Побудуйте числовий рейтинг. Вимоги:

- використати **щонайменше три** різні сигнали;
- звести різнорідні величини до спільної шкали (підказка: `rank(pct=True)`);
- **пояснити у звіті, чому саме такі ваги**.

In [55]:
```
metric = cand.to_crs(32635)
cand['area_m2'] = np.where(metric.geometry.geom_type.isin(['Polygon', 'Multi
                           metric.area.values, 0.0)

wcols = [c for c in ('wikidata', 'wikipedia') if c in cand]
cand['wiki'] = cand[wcols].notna().any(axis=1).astype(int) if wcols else 0

lang_cols = [c for c in cand.columns if c.startswith('name:')]
cand['n_lang'] = cand[lang_cols].notna().sum(axis=1) if lang_cols else 0
```

```
cand['score'] = (
    0.5 * cand['wiki'].rank(pct=True) +
    0.35 * cand['n_lang'].rank(pct=True) +
    0.15 * cand['area_m2'].rank(pct=True)
)

print('кандидатів         :', len(cand))
print('мають wiki         :', int(cand['wiki'].sum()))
print('мають n_lang > 2   :', int(cand['n_lang'].gt(2).sum()))
print('мають score > 0.6  :', int(cand['score'].gt(0.6).sum()))
```

```
кандидатів         : 2156
мають wiki         : 180
мають n_lang > 2   : 305
мають score > 0.6  : 417
```

Завдання 3.2 — обов'язкове порівняння

Побудуйте **другий** рейтинг за найпростішим критерієм — площею будівлі — і порахуйте, скільки об'єктів збігається в перших десяти.

Це головне питання роботи: обидва списки отримані з тих самих даних, тим самим кодом, без жодної помилки. Якщо вони різні — поясніть у звіті, що це означає для слова «найкращі».

In [56]:
```
N_SMALL, N_LARGE = 10, 30

top_score = cand.nlargest(N_LARGE, 'score').reset_index(drop=True)
top10     = top_score.head(N_SMALL).reset_index(drop=True)
top_naive = cand.nlargest(N_SMALL, 'area_m2').reset_index(drop=True)

overlap = len(set(top10['name']) & set(top_naive['name']))
print('перетин: %d з %d' % (overlap, N_SMALL))

display(pd.DataFrame({'за вашим критерієм': list(top10['name']),
                      'за площею': list(top_naive['name'])}))
```

```
перетин: 2 з 10
```

	за вашим критерієм	за площею
0	Софійський монастир	Китаївський археологічний комплекс
1	Національний цирк України	Києво-Печерська цитадель
2	Михайлівський Золотоверхий монастир	Бабин Яр
3	Маріїнський палац	Київська фортеця
4	Софійський собор	Державний музей авіації
5	Міжнародний центр культури і мистецтв	Недобудований асфальтовий завод
6	Національний художній музей України	Парк воїнів-інтернаціоналістів
7	Арка Свободи українського народу	Софійський монастир
8	Національний музей Тараса Шевченка	Михайлівський Золотоверхий монастир
9	Золоті ворота	Городище XII

Завдання 3.3 — дублікати

OSM наповнюють різні люди. Той самий музей трапляється двічі під різними назвами, а вежа буває окремим об'єктом усередині палацу. Знайдіть у `top_score` пари, що стоять ближче **60 метрів** одна до одної, і вирішіть, що з ними робити.

Рішення (лишити / злити / видалити) опишіть у звіті.

In [57]:
```
def close_pairs(gdf, limit_m=60):
    p = gdf.to_crs(32635).geometry.representative_point()
    XY = np.c_[p.x.values, p.y.values]
    D = np.sqrt(((XY[:, None, :] - XY[None, :, :]) ** 2).sum(-1))
    np.fill_diagonal(D, np.inf)
    out = [(i, j, D[i, j]) for i in range(len(XY)) for j in range(i + 1, ler
           if D[i, j] < limit_m]
    return sorted(out, key=lambda t: t[2])

pairs = close_pairs(top_score)
print('пар ближче 60 м:', len(pairs))
for i, j, d in pairs:
    print('%5.1f м | %-38s <-> %-38s | score %.3f / %.3f'
          % (d, str(top_score['name'].iloc[i])[:38],
             str(top_score['name'].iloc[j])[:38],
             top_score['score'].iloc[i], top_score['score'].iloc[j]))

def row_info(i):
    r = top_score.iloc[i]
    rid = r['id'] if 'id' in top_score else r.get('osmid', '')
    wik = 'є' if r.get('wiki', 0) else '—'
    return '%s | id %s | wiki %s | площа %.0f | score %.3f' % (
        str(r['name']), rid, wik, r['area_m2'], r['score'])
```

```
for i, j, d in pairs:
    print('--- %.1f м ---' % d)
    print('   ', row_info(i))
    print('   ', row_info(j))

top_score = top_score[top_score['id'] != 5012711].reset_index(drop=True)
top10     = top_score.head(N_SMALL).reset_index(drop=True)
print('у top_score лишилось:', len(top_score))
```

```
пар ближче 60 м: 1
 30.6 м | Софійський монастир                    <-> Софійський собор
| score 0.978 / 0.977
--- 30.6 м ---
    Софійський монастир | id 5012711 | wiki є | площа 49621 | score 0.978
    Софійський собор | id 27830282 | wiki є | площа 2359 | score 0.977
у top_score лишилось: 29
```

---

4. Старт: вокзал, і прив'язка точок до графа

In [58]:
```
print('станції у шарі:', len(stations))
print(stations[['name', 'railway', 'building']].dropna(how='all').head(12).t
print()
print(stations[['name', 'building']][stations['railway'] == 'station'].dropr
```

```
станції у шарі: 149
                         name  railway building
0   Київ-Пасажирський-Дитячий  station      NaN
1               Проспект Науки     halt      NaN
2                       Алмаз     halt      NaN
3                Дарниця-Депо     halt      NaN
4                        ДВРЗ     halt      NaN
5               Караваєві Дачі     halt      NaN
6                    Заводська  station      NaN
7                  Протасів Яр     halt      NaN
8             Київ-Деміївський  station      NaN
9                  Лівобережна     halt      NaN
10                     Дарниця  station      NaN
11                  Лісництво     halt      NaN

                         name building
0   Київ-Пасажирський-Дитячий      NaN
6                    Заводська      NaN
8             Київ-Деміївський      NaN
10                     Дарниця      NaN
14              Петро Кривоніс      NaN
17         Імені Георгія Кірпи      NaN
18         Микільська Слобідка      NaN
19                     Почайна      NaN
22           Київ-Пасажирський      NaN
25                 Лівобережна      NaN
26                   Гідропарк      NaN
27                     Дарниця      NaN
```

In [59]:
```
station = stations[stations['name'] == 'Київ-Пасажирський']
print('старт:', station['name'].iloc[0])
```

```
старт: Київ-Пасажирський
```

In [60]:
```
G = nx.DiGraph()
for t in nodes.itertuples():
    G.add_node(t.osmid, x=t.x, y=t.y)
for t in edges.itertuples():
    L = float(t.length)
    if G.has_edge(t.u, t.v):
        G[t.u][t.v]['length'] = min(G[t.u][t.v]['length'], L)
    else:
        G.add_edge(t.u, t.v, length=L)
print('граф: %d вузлів, %d ребер' % (G.number_of_nodes(), G.number_of_edges(

NODES_M = nodes.to_crs(32635)
NODE_XY = np.c_[NODES_M.geometry.x.values, NODES_M.geometry.y.values]
NODE_ID = nodes['osmid'].values

def snap(points_gdf):
    """Найближчий вузол графа до кожної точки. Рахуємо в метричній CRS."""
    pmet = points_gdf.to_crs(32635)
    ids, dist = [], []
    for p in pmet.geometry:
        d2 = (NODE_XY[:, 0] - p.x) ** 2 + (NODE_XY[:, 1] - p.y) ** 2
        k = int(np.argmin(d2))
        ids.append(NODE_ID[k]); dist.append(float(np.sqrt(d2[k])))
    return np.array(ids), np.array(dist)

def make_points(top):
    names = ['СТАРТ: ' + str(station['name'].iloc[0])] + list(top['name'])
    geoms = [station.geometry.iloc[0].representative_point()] + \
            [g.representative_point() for g in top.geometry]
    return gpd.GeoDataFrame({'name': names}, geometry=geoms, crs='EPSG:4326'
```

```
граф: 178332 вузлів, 514718 ребер
```

Завдання 4.1 — обов'язкова перевірка

Напишіть `check_snap`, яка повідомляє:

- скільки точок і скільки **різних** вузлів вони зайняли;
- медіану й максимум відстані «точка → вузол»;
- якщо вузлів менше, ніж точок — **назви точок, що злиплись**.

Контрольна ознака: на пішохідній мережі різних вузлів має бути **стільки ж, скільки точок**. Якщо менше — ви взяли автомобільну мережу або місто надто мале; зафіксуйте це у звіті, бо далі такі пари отримають нульову віддаль.

In [61]:
```
def check_snap(points, ids, dist, label):
    n_pts = len(points)
    n_nodes = len(set(ids))
```

```
    print('[%s] точок: %d | різних вузлів: %d' % (label, n_pts, n_nodes))
    print('        точка->вузол: медіана %.1f м, максимум %.1f м'
          % (np.median(dist), np.max(dist)))
    if n_nodes < n_pts:
        groups = collections.defaultdict(list)
        for name, nid in zip(points['name'], ids):
            groups[nid].append(str(name))
        for nid, names in groups.items():
            if len(names) > 1:
                print('        ЗЛИПЛИСЬ -> вузол %s: %s' % (nid, ' | '.join(
    else:
        print('        усі точки на різних вузлах')
```

---

5. Матриця віддалей

Дві матриці: **пряма лінія** і **дорогою**. Їхнє відношення — коефіцієнт обходу — показує, наскільки пряма лінія бреше.

Завдання 5.1

In [62]:
```
def road_matrix(ids):
    """Матриця найкоротших віддалей графом між заданими вузлами."""
    k = len(ids)
    D = np.full((k, k), np.inf)
    for i, s in enumerate(ids):
        lengths = nx.single_source_dijkstra_path_length(G, s, weight='length
        for j, t in enumerate(ids):
            D[i, j] = lengths.get(t, np.inf)
    np.fill_diagonal(D, 0.0)
    return D

def straight_matrix(points):
    p = points.to_crs(32635).geometry.representative_point()
    XY = np.c_[p.x.values, p.y.values]
    return np.sqrt(((XY[:, None, :] - XY[None, :, :]) ** 2).sum(-1))

def detour_factor(D, E):
    ratios = [
        D[i, j] / E[i, j]
        for i in range(len(D)) for j in range(len(D))
        if i != j and np.isfinite(D[i, j]) and E[i, j] > 0
    ]
    return float(np.median(ratios)) if ratios else float('nan')
```

Контрольні ознаки — звіртесь, не підганяючи:

- коефіцієнт обходу для пішохідної мережі у місті зазвичай **1,2–1,6**;
- якщо у вас рівно **1,0** — ви порівнюєте матрицю саму із собою;
- якщо **більше 2** — найімовірніше взято автомобільну мережу з односторонніми вулицями, або місто має річку/залізницю, яку нема де

перетнути. Друге — цікавий результат, перше — помилка. Розберіться, що саме у вас, і напишіть це.

Якщо серед віддалей трапились `inf` — частина графа не зв'язана з рештою. Не замінюйте `inf` великим числом мовчки: спершу з'ясуйте, які це точки.

In [63]:
```
pts10 = make_points(top10)
ids10, dist10 = snap(pts10)
check_snap(pts10, ids10, dist10, 'мала задача')

t0 = time.time(); D10 = road_matrix(ids10); print('матриця за %.3f с' % (tim
E10 = straight_matrix(pts10)
print('недосяжних пар    :', int(np.isinf(D10).sum()))
print('асиметрія         : %.0f м' % float(np.max(np.abs(D10 - D10.T))))
print('коефіцієнт обходу : %.2f' % detour_factor(D10, E10))
```

```
[мала задача] точок: 11 | різних вузлів: 11
        точка->вузол: медіана 27.8 м, максимум 46.2 м
        усі точки на різних вузлах
матриця за 6.491 с
недосяжних пар    : 0
асиметрія         : 0 м
коефіцієнт обходу : 1.25
```

> **Асиметрія.**
> D_{ij}
> $neq D_{ji}$
> — не помилка: односторонній рух робить шлях туди і назад різним.
> На пішохідній мережі асиметрія мала, на автомобільній — помітна.
> Подивіться на своє число і поясніть його у звіті.

---

6. Задача комівояжера

метод	що робить	складність
найближчий сусід	щоразу йде в найближчу невідвідану точку	$O(n^2)$
2-opt	розвертає ділянки готового маршруту, поки коротшає	$O(n^2)$ за прохід
Held–Karp	динамічне програмування за підмножинами — **точний**	$O(2^n n^2)$ часу, $O(2^n n)$ пам'яті

Held–Karp дано готовим: він не є метою роботи, але без нього ви не дізнаєтесь, **наскільки** помиляються перші два.

Завдання 6.1 — реалізуйте перші два методи

In [64]:
```
def tour_length(D, tour):
    return float(sum(D[tour[i], tour[i + 1]] for i in range(len(tour) - 1)))

def nearest_neighbour(D):
    k = len(D)
    unvisited = set(range(1, k))
    tour = [0]
    while unvisited:
        last = tour[-1]
        nxt = min(unvisited, key=lambda j: D[last, j])
        tour.append(nxt)
        unvisited.discard(nxt)
    tour.append(0)
    return tour

def two_opt(D, tour):
    tour = tour[:]
    n = len(tour)
    improved = True
    while improved:
        improved = False
        for i in range(1, n - 2):
            for j in range(i + 1, n - 1):
                a, b = tour[i - 1], tour[i]
                c, d = tour[j], tour[j + 1]
                delta = D[a, c] + D[b, d] - D[a, b] - D[c, d]
                if delta < -1e-12:
                    tour[i:j + 1] = tour[i:j + 1][::-1]
                    improved = True
    return tour
```

In [65]:
```
def held_karp(D):
    """Точний розв'язок. Вузол 0 — старт і фініш. ДАНО, писати не треба."""
    k = len(D); N = k - 1; INF = float('inf')
    dp = [[INF] * N for _ in range(1 << N)]
    par = [[-1] * N for _ in range(1 << N)]
    for j in range(N):
        dp[1 << j][j] = D[0, j + 1]
    for S in range(1 << N):
        for j in range(N):
            cur = dp[S][j]
            if cur == INF or not (S >> j) & 1:
                continue
            for t in range(N):
                if (S >> t) & 1:
                    continue
                S2 = S | (1 << t); v = cur + D[j + 1, t + 1]
                if v < dp[S2][t]:
                    dp[S2][t] = v; par[S2][t] = j
    full = (1 << N) - 1
    bj = min(range(N), key=lambda j: dp[full][j] + D[j + 1, 0])
    best = dp[full][bj] + D[bj + 1, 0]
    path = []; S = full; j = bj
    while j != -1:
```

```
        path.append(j + 1); pj = par[S][j]; S ^= (1 << j); j = pj
    return [0] + path[::-1] + [0], float(best)
```

Самоперевірка

Ці твердження мають виконатись для **будь-якого** міста. Якщо хоч одне впало — шукайте помилку в коді, а не в даних.

In [66]:
```
def check_tour(tour, k, label):
    assert tour[0] == 0 and tour[-1] == 0, '%s: маршрут має починатись і зак
    assert sorted(tour[:-1]) == list(range(k)), '%s: кожна точка рівно один
    print('%s: структура маршруту правильна (%d точок)' % (label, k))

nn10 = nearest_neighbour(D10)
op10 = two_opt(D10, nn10)
ex10, L_opt = held_karp(D10)

for t, lbl in [(nn10, 'найближчий сусід'), (op10, '2-opt'), (ex10, 'Held-Kar
    check_tour(t, len(ids10), lbl)

assert tour_length(D10, op10) <= tour_length(D10, nn10) + 1e-6, '2-opt не мо
assert L_opt <= tour_length(D10, op10) + 1e-6, 'точний розв\u2019язок не мож
print()
print('усі перевірки пройдено')
```

```
найближчий сусід: структура маршруту правильна (11 точок)
2-opt: структура маршруту правильна (11 точок)
Held-Karp: структура маршруту правильна (11 точок)

усі перевірки пройдено
```

In [67]:
```
rows = []
for name, tour in [('найближчий сусід', nn10), ('2-opt', op10), ('Held-Karp
    L = tour_length(D10, tour)
    rows.append({'метод': name, 'довжина, м': round(L),
                 'гірше оптимуму, %': round(100 * (L / L_opt - 1), 2)})
display(pd.DataFrame(rows).set_index('метод'))

print()
print('оптимальний порядок:')
for step, i in enumerate(ex10):
    print('  %2d. %s' % (step, pts10['name'].iloc[i]))
print()
print('довжина: %.2f км' % (L_opt / 1000))
```

метод	довжина, м	гірше оптимуму, %
найближчий сусід	16225	12.01
2-opt	14768	1.95
Held-Karp (точний)	14486	0.00

```
оптимальний порядок:
   0. СТАРТ: Київ-Пасажирський
   1. Національний цирк України
   2. Золоті ворота
   3. Софійський собор
   4. Михайлівський Золотоверхий монастир
   5. Музей «Чорнобиль»
   6. Арка Свободи українського народу
   7. Національний художній музей України
   8. Маріїнський палац
   9. Міжнародний центр культури і мистецтв
  10. Національний музей Тараса Шевченка
  11. СТАРТ: Київ-Пасажирський

довжина: 14.49 км
```

Завдання 6.2 — велика задача

Спершу оцініть, чи можливий тут точний розв'язок. Порахуйте для $n = 11, 16, 21, 26, 31$ розмір таблиці Held–Karp ($2^{n-1} cdot (n-1)$ клітин) і скільки це гігабайтів при 8 байтах на клітину.

In [68]:
```
rows = []
for n in [11, 16, 21, 26, 31]:
    cells = 2 ** (n - 1) * (n - 1)
    rows.append({'n': n, 'клітин': cells, 'ГіБ (8 Б на клітину)': cells * 8
display(pd.DataFrame(rows).set_index('n'))
```

n	клітин	ГіБ (8 Б на клітину)
11	10240	0.000076
16	491520	0.003662
21	20971520	0.156250
26	838860800	6.250000
31	32212254720	240.000000

Подивіться на останній рядок і напишіть у звіті, **чому саме** точний розв'язок на 30 точках недоступний. Слово «довго» відповіддю не є.

In [69]:
```
pts30 = make_points(top_score)
ids30, dist30 = snap(pts30)
check_snap(pts30, ids30, dist30, 'велика задача')

D30 = road_matrix(ids30)
E30 = straight_matrix(pts30)
```

```
print('коефіцієнт обходу: %.2f' % detour_factor(D30, E30))

nn30 = nearest_neighbour(D30)
t0 = time.time(); op30 = two_opt(D30, nn30); t_op30 = time.time() - t0
check_tour(op30, len(ids30), '2-opt на 30 точках')

L_nn30, L_op30 = tour_length(D30, nn30), tour_length(D30, op30)
print()
print('найближчий сусід : %8.0f м' % L_nn30)
print('2-opt            : %8.0f м  за %.3f с  (коротше на %.1f%%)'
      % (L_op30, t_op30, 100 * (1 - L_op30 / L_nn30)))
print()
print('ЦЕ — база порівняння для лабораторної роботи № 8.')
```

```
[велика задача] точок: 30 | різних вузлів: 30
        точка->вузол: медіана 19.1 м, максимум 84.9 м
        усі точки на різних вузлах
коефіцієнт обходу: 1.23
2-opt на 30 точках: структура маршруту правильна (30 точок)

найближчий сусід :    44842 м
2-opt            :    41691 м  за 0.000 с  (коротше на 7.0%)

ЦЕ — база порівняння для лабораторної роботи № 8.
```

---

7. Просторова індексація: H3

H3 — ієрархічна сітка шестикутників. Рівень 9 — приблизно 200 м по ребру.

📖 https://h3geo.org/docs/core-library/restable/

Завдання 7.1

Розкладіть **усіх** кандидатів по комірках H3 рівня 9 і побудуйте картограму щільності. На тій самій карті покажіть межу міста і точки вашого маршруту.

In [70]:
```
import h3
from matplotlib.colors import to_hex
from shapely.geometry import Polygon

H3_RES = 9
print('середнє ребро комірки: %.0f м' % h3.average_hexagon_edge_length(H3_RE

pts = cand.geometry.representative_point()
cand['h3'] = [h3.latlng_to_cell(p.y, p.x, H3_RES) for p in pts]

dens = cand['h3'].value_counts()
hexes = gpd.GeoDataFrame(
    {'h3': dens.index, 'count': dens.values},
    geometry=[Polygon([(lng, lat) for lat, lng in h3.cell_to_boundary(c)]) f
    crs='EPSG:4326')
```

```
vmax = int(hexes['count'].max())
fmap = folium.Map(location=[pts.y.mean(), pts.x.mean()], zoom_start=12, tile

folium.GeoJson(
    hexes,
    style_function=lambda f: {'fillColor': to_hex(plt.cm.YlOrRd(0.15 + 0.85
                              'color': '#555', 'weight': 0.3, 'fillOpacity':
    tooltip=folium.GeoJsonTooltip(fields=['count'], aliases=["пам'яток:"]),
).add_to(fmap)

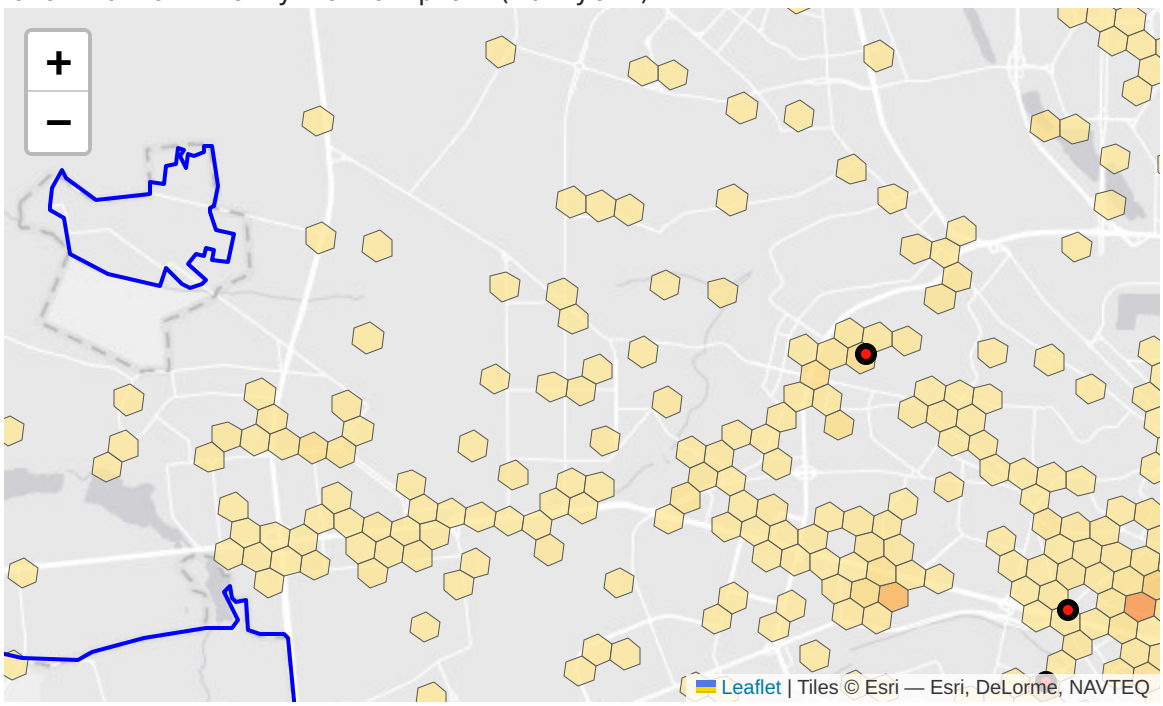
folium.GeoJson(boundary.to_crs(4326),
               style_function=lambda f: {'fill': False, 'color': 'blue', 'we

for _, r in pts30.to_crs(4326).iterrows():
    folium.CircleMarker(location=[r.geometry.y, r.geometry.x], radius=4,
                        color='black', fill=True, fill_color='red', fill_opa
                        tooltip=r['name']).add_to(fmap)

s = dens.sort_values(ascending=False)
half = s.sum() / 2
n_half = int((s.cumsum() < half).sum()) + 1
print("комірок із даними  :", len(s))
print("половина пам'яток у %d комірках (%.0f%% усіх)" % (n_half, 100 * n_hal
fmap.save('lr5_m1.html')
fmap
```

```
середнє ребро комірки: 201 м
комірок із даними  : 747
половина пам'яток у 76 комірках (10% усіх)
```

Out[70]:



> **Пастка.** `h3.cell_to_boundary` повертає пари **(lat, lng)**, а `shapely.Polygon` чекає **(x, y)**, тобто (lng, lat). Переплутаєте — місто опиниться в Індійському океані. Це видно одразу, тому дивіться на карту.

Питання до звіту

У скількох комірках зосереджено половину всіх пам'яток вашого міста? Що це говорить про форму маршруту?

---

8. Просторове поєднання

Завдання 8.1

Для кожної точки маршруту знайдіть найближчий заклад із шару `amenities` (`sjoin_nearest`) і виведіть таблицю з відстанню в метрах.

Нагадування: `sjoin_nearest` рахує в одиницях CRS. У градусах результат буде безглуздим — перепроєктуйте обидва шари в метричну систему.

In [71]:
```
def nearest_amenity(points):
    p = points[['name', 'geometry']].rename(columns={'name': 'точка'}).to_cr
    a = amenities.to_crs(32635)
    a = a.rename(columns={c: {'name': 'заклад', 'amenity': 'тип'}[c]
                          for c in ('name', 'amenity') if c in a})
    a = a[[c for c in ('заклад', 'тип', 'geometry') if c in a]]
    j = gpd.sjoin_nearest(p, a, how='left', distance_col='відстань, м')
    j = j.sort_values('відстань, м').groupby(level=0).head(1)
    cols = [c for c in ('точка', 'заклад', 'тип', 'відстань, м') if c in j]
    return j[cols].reset_index(drop=True)

nearest = nearest_amenity(pts10)
nearest['відстань, м'] = nearest['відстань, м'].round(1)
display(nearest)
```

	точка	заклад	тип	відстань, м
0	СТАРТ: Київ-Пасажирський	Loven bistro	cafe	13.5
1	Національний художній музей України	Сулугуни	restaurant	50.2
2	Золоті ворота	Porta D'oro	restaurant	50.8
3	Михайлівський Золотоверхий монастир	Кав'ярня «Благокава»	cafe	51.0
4	Музей «Чорнобиль»	Coffee Dream	cafe	62.5
5	Софійський собор	NaN	toilets	64.5
6	Національний цирк України	Моменто	restaurant	68.0
7	Арка Свободи українського народу	NaN	toilets	100.5
8	Національний музей Тараса Шевченка	Bubble	cafe	109.4
9	Міжнародний центр культури і мистецтв	Solo Pizza	restaurant	121.9
10	Маріїнський палац	5th Ave Residence	restaurant	147.1

---

9. Маршрут на карті

Лінія маршруту — це **геометрія ребер графа**, а не відрізки між пам'ятками.

Завдання 9.1

Побудуйте карту, на якій **обидві** лінії: справжній маршрут по ребрах і прямі відрізки між точками. Різниця має бути видно оком.

In [72]:
```
node_xy = nodes.set_index('osmid')[['x', 'y']]

def route_coords(tour, ids):
    """Координати маршруту по ребрах графа у форматі (lat, lon) для folium."
    coords = []
    for a, b in zip(tour[:-1], tour[1:]):
        try:
            path = nx.shortest_path(G, ids[a], ids[b], weight='length')
        except nx.NetworkXNoPath:
            path = [ids[a], ids[b]]
        for nd in path:
            coords.append((float(node_xy.loc[nd, 'y']), float(node_xy.loc[nd
    return coords
```

```
fmap = folium.Map(location=[pts10.geometry.y.mean(), pts10.geometry.x.mean()
                  zoom_start=15, tiles=BASEMAP)

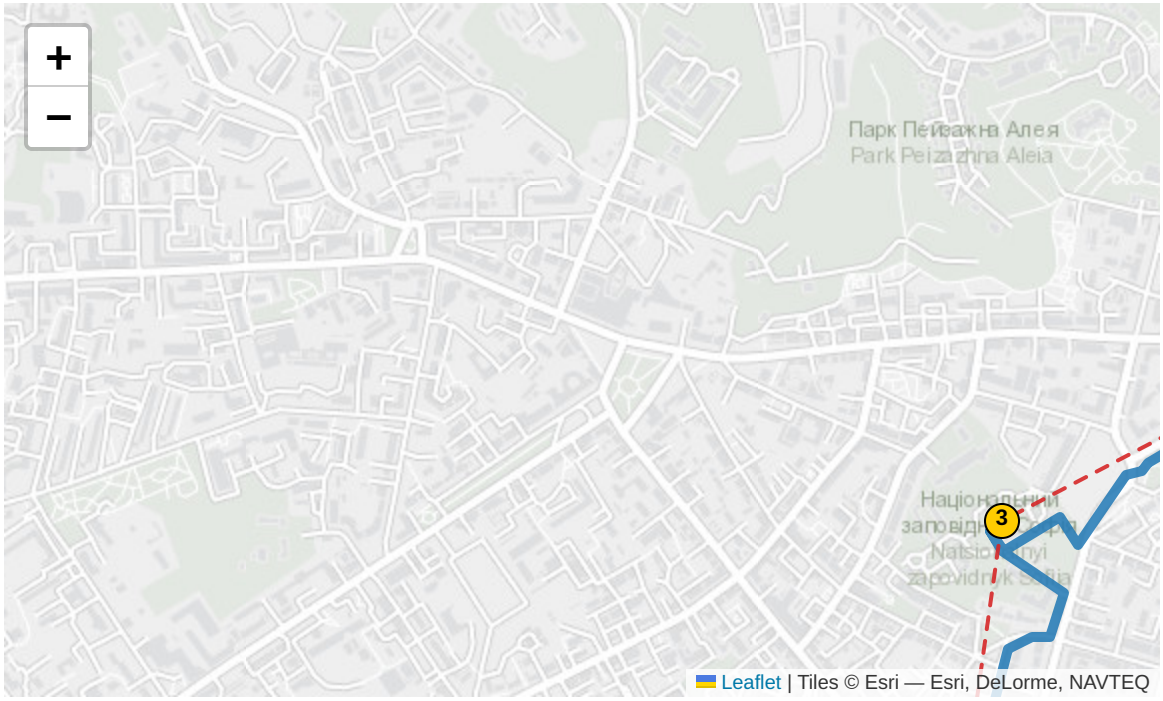
folium.PolyLine(route_coords(ex10, ids10), color='#1f77b4', weight=5, opacit
                tooltip='маршрут по ребрах графа').add_to(fmap)

straight = [(float(pts10.geometry.y.iloc[i]), float(pts10.geometry.x.iloc[i]
folium.PolyLine(straight, color='#d62728', weight=2, opacity=0.9, dash_array
                tooltip='прямі відрізки між точками').add_to(fmap)

for step, i in enumerate(ex10):
    p = pts10.geometry.iloc[i]
    folium.Marker(
        location=[p.y, p.x],
        icon=folium.DivIcon(
            icon_size=(18, 18), icon_anchor=(9, 9),
            html=('<div style="background:#ffcc00;border:1px solid #000;borc
                  'width:18px;height:18px;line-height:18px;text-align:center
                  'font:700 11px sans-serif;color:#000">%d</div>') % step),
        tooltip='%d. %s' % (step, pts10['name'].iloc[i]),
    ).add_to(fmap)

fmap.save('lr5_маршрут.html')
fmap
```

Out[72]:



---

10. Результати для лабораторної роботи № 8

Збережіть файл, який ЛР8 прочитає і спробує побити.

In [73]:
```
result = {
    'студент': STUDENT_NAME,
    'номер': LIST_NUMBER,
    'місто': CITY,
    'мала_задача': {
        'точок': len(ids10),
        'оптимум_м': round(L_opt, 1),
        'оптимальний_порядок': [str(pts10['name'].iloc[i]) for i in ex10],
        'найближчий_сусід_м': round(tour_length(D10, nn10), 1),
        'двоопт_м': round(tour_length(D10, op10), 1),
    },
    'велика_задача': {
        'точок': len(ids30),
        'найближчий_сусід_м': round(L_nn30, 1),
        'двоопт_м': round(L_op30, 1),
        'двоопт_с': round(t_op30, 4),
        'точки': [{'name': str(pts30['name'].iloc[i]),
                   'lat': float(pts30.geometry.y.iloc[i]),
                   'lon': float(pts30.geometry.x.iloc[i])} for i in range(le
        'матриця_віддалей_м': [[None if not np.isfinite(v) else round(float(
                               for v in row] for row in D30],
    },
}
with open('lr5_результати.json', 'w', encoding='utf-8') as f:
    json.dump(result, f, ensure_ascii=False, indent=1)
print('збережено lr5_результати.json (%.1f КБ)' % (os.path.getsize('lr5_резу
```

```
збережено lr5_результати.json (15.5 КБ)
```

---

11. Звіт

Відповіді пишіть просто в наступній комірці, числами зі своїх обчислень.

1. **Скільки об'єктів ви відкинули** на кроці 2 і за яким правилом? Чи погодились би ви віддати цей фільтр замовнику без пояснень?
2. **Які сигнали ввійшли у ваш рейтинг і з якими вагами?** Чому саме так?
3. **Скільки об'єктів збіглося** у ваших двох рейтингах із десяти? Що це означає для слова «найкращі»?
4. **Який у вас коефіцієнт обходу** і чи типовий він? Якщо ні — що в місті його пояснює?
5. **На скільки відсотків** найближчий сусід гірший за точний розв'язок? Чи здивувало вас це число?
6. **Чому точний розв'язок на 30 точках недоступний?** Назвіть конкретну величину, яка цьому заважає.
7. **Що у ваших даних виявилось неправильним** — дублікати, пропущені назви, об'єкти не на місці? Як ви з цим вчинили?

Декларація про використання штучного інтелекту

Зазначте, чи користувались ШІ-помічником, у яких саме місцях і як перевіряли отриманий код. Використання дозволене; **неперевірене** використання — ні.

In [75]:
```
REPORT = '''
1. відкинув 593 об'єкти без name. без пояснень не погодився би
2. сигнали wiki 0.5, кількість мов для name 0.35, площа 0.15, бо об'єкти зі
3. 2 з 10
4. коефіцієнт обходу 1.25, типовий для великого міста
5. NN гірший за оптимум на 12.01%, не здивувало, бо NN жадібний
6. бо таблиця held-karp розростається до 240 гб через множник 2^n
7. один частковий дублікат (софійський монастир/собор) та об'єкти без імен

Декларація про використання ШІ: використовував для коду, висновки робив сам
'''
print(REPORT)
```

```
1. відкинув 593 об'єкти без name. без пояснень не погодився би
2. сигнали wiki 0.5, кількість мов для name 0.35, площа 0.15, бо об'єкти зі
сторінками в вікіпедії зазвичай значущі, а кількість мов означає, що об'єкт
цікавий людям по всьому світу
3. 2 з 10
4. коефіцієнт обходу 1.25, типовий для великого міста
5. NN гірший за оптимум на 12.01%, не здивувало, бо NN жадібний
6. бо таблиця held-karp розростається до 240 гб через множник 2^n
7. один частковий дублікат (софійський монастир/собор) та об'єкти без імен

Декларація про використання ШІ: використовував для коду, висновки робив сам
```